Chapter 9 — Unsupervised Learning

Unsupervised learning works with **no labels**: the goal is to find structure in the data itself. The four big jobs (each algorithm is explained in its own section):

Job	Question it answers	Main algorithms
Clustering	Which examples belong together?	k-means, hierarchical, DBSCAN, Gaussian mixtures
Dimensionality reduction	Can I describe the data with fewer numbers, or see it in 2-D?	PCA, t-SNE, UMAP
Anomaly detection	Which examples don't fit?	z-score, Isolation Forest, One-Class SVM, LOF
Association rules	Which items occur together?	Apriori, FP-Growth

The hard part is always **evaluation** — without labels, what is "good"? — covered last.

What you'll learn: k-means, k-means++, elbow and silhouette, k-means' limits; agglomerative clustering and dendrograms; DBSCAN; Gaussian mixtures and EM; PCA; t-SNE and UMAP; anomaly detection (statistics, Isolation Forest, One-Class SVM, LOF); Apriori; evaluating unsupervised models.

All code is in `code/`, figures in `img/`; outputs are real (scikit-learn 1.9, SciPy 1.18).

9.1 k-means clustering

What it is

k-means splits data into k **clusters**, each point belonging to the nearest **centroid** (cluster mean); you choose k. Like placing k pizza shops so every household walks to its nearest one with minimal total walking: move each shop to the middle of its customers, reassign, repeat.

Why it matters

Simple, fast (~$O(n \cdot k \cdot d)$ per iteration: points × clusters × features), scales to millions, and the default first clustering method: customer segmentation, document grouping, colour quantisation (compressing an image's colours), "visual words" or **codebooks** (fixed lists of representative patterns), initialising other models.

How it works (Lloyd's algorithm)

k-means minimises **inertia** (within-cluster sum of squares, **WCSS**): total squared distance from points to their centroids.

1. **Initialise** k centroids (random points or k-means++).
2. **Assign** each point to its nearest centroid.
3. **Update** each centroid to the mean of its points.
4. **Repeat** 2–3 until assignments stop changing.

Each step can only lower inertia, so it converges — to a **local** minimum (no small change improves it, but a better solution may exist) that depends on the start.

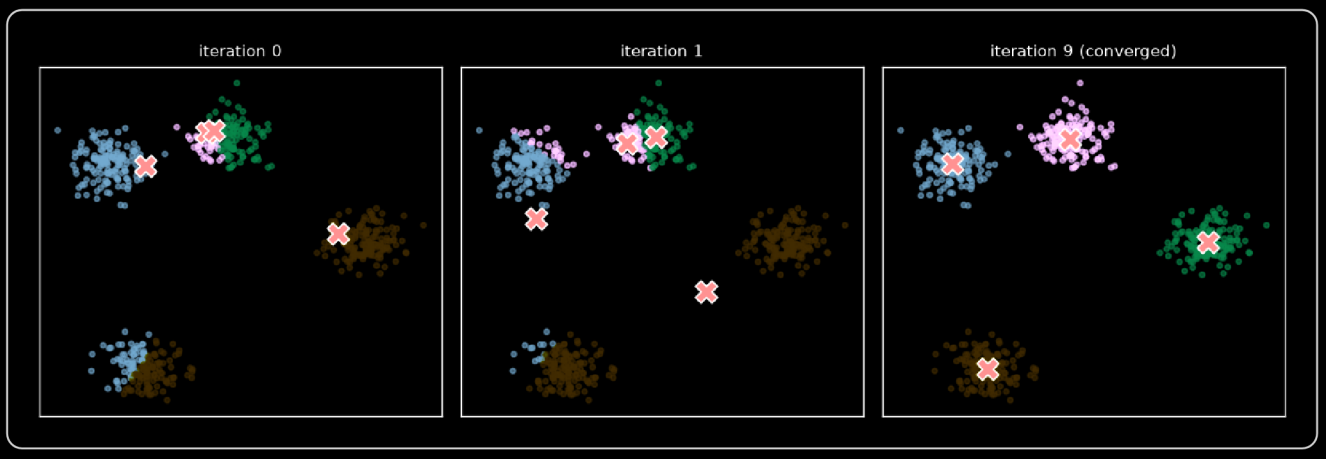
Worked example (1-D). Points 1, 2, 3, 10, 11, 12; k = 2; start at 1 and 2.

- Assign {1} and {2, 3, 10, 11, 12}; update $c_1 = 1$, $c_2 = 7.6$.
- 3 is 2 from c_1 and 4.6 from c_2 → {1, 2, 3}, {10, 11, 12}; update $c_1 = 2$, $c_2 = 11$.
- No change → converged. Inertia = (1 + 0 + 1) + (1 + 0 + 1) = **4**.

From scratch (`code/kmeans_demo.py`):

```python
def kmeans_scratch(X, k, rng, max_iter=100):
    centroids = X[rng.choice(len(X), k, replace=False)]          # 1. random init
    for it in range(max_iter):
        d = np.linalg.norm(X[:, None, :] - centroids[None, :, :], axis=2)
        labels = d.argmin(axis=1)                                 # 2. assign to nearest centroid
        new = np.array([X[labels == j].mean(axis=0) for j in range(k)])  # 3. move centroids to means
        if np.allclose(new, centroids):                           # 4. stop when nothing moves
            break
        centroids = new
    inertia = ((X - centroids[labels]) ** 2).sum()
    return labels, centroids, inertia
```

```
from scratch: converged in 9 iterations, inertia = 1160
sklearn     : inertia = 1160
```



Two starting centroids fell in one blob and another blob had none; it recovered in 9 iterations — not always that lucky.

k-means++ initialisation

k-means++ (Arthur & Vassilvitskii, 2007) spreads initial centroids:

1. First centroid uniformly at random.
2. For each point, D(x) = distance to the nearest chosen centroid.
3. Next centroid drawn with probability **∝ D(x)²** — far points much more likely.
4. Repeat to k, then run normal k-means.

Guarantee: expected inertia within O(log k) of optimal. On 8 well-separated blobs, one run per seed:

```
init          inertia over 20 single-run seeds, 8 true clusters (min / mean / max)
random            1673 /     9002 /    32908
k-means++          704 /      704 /      704
```

Random init never found the answer in 20 tries; k-means++ every time. scikit-learn defaults to `init="k-means++"`, and `n_init` restarts keep the best run.

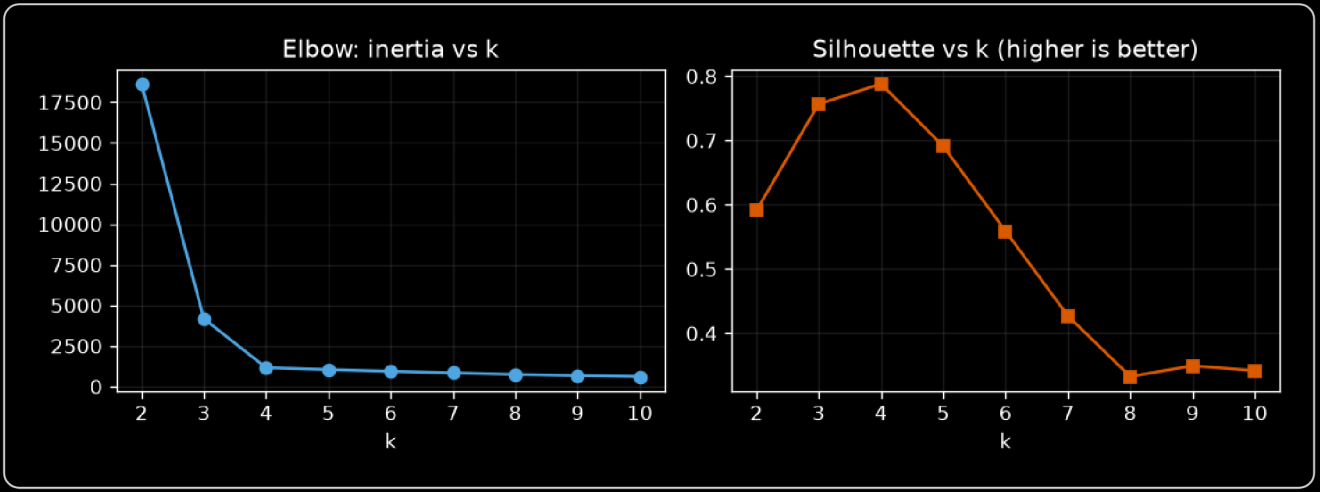
Choosing k: elbow and silhouette

Elbow method: plot inertia vs k (it always falls; k = n gives 0) and look for the bend where an extra cluster stops helping.

Silhouette per point: a = mean distance to its own cluster (cohesion), b = mean distance to the **nearest other** cluster (separation), s = (b − a) / max(a, b) ∈ [−1, 1]. s ≈ 1 well inside, ≈ 0 on a border, < 0 probably misassigned; the **silhouette score** averages s. In the 1-D example point 3 has a = 1.5, b = 8, s = 0.81.

On 4 true blobs:

k	inertia	silhouette
2	18638	0.591
3	4179	0.757
4	1160	0.788
5	1046	0.691
6	938	0.559



Both point to k = 4. On real data the elbow is often vague — prefer silhouette, domain knowledge ("marketing can handle 5 segments") and stability across runs.

Limits of k-means

It assumes **round (spherical), similar-sized clusters separated by straight boundaries**. It fails on:

- **Non-convex shapes** (moons, rings) — §9.4 comparison.
- **Elongated or differently spread clusters** — a diffuse cluster gets cut up.
- **Outliers** (drag the means); **unscaled features** (always standardise).
- **Categorical data** — means are meaningless (use k-modes or k-prototypes).
- You must choose k.

Common mistakes

- Not scaling; trusting one random-init run.
- Running the elbow on data with no clusters — k-means returns k groups even from uniform noise.

In the interview

30-second answer: "k-means picks k centroids, assigns each point to the nearest, moves each centroid to the mean of its points, and repeats until nothing changes. It minimises within-cluster squared distance and converges to a local minimum, so we use k-means++ initialisation and multiple restarts. We choose k with the elbow or silhouette score. It assumes round, similar-sized clusters and needs scaled features."

- *Why k-means++?* Spread starts ($\propto$ squared distance) → faster, better, more consistent results.
- *Complexity?* O(n·k·d) per iteration; **Mini-Batch k-means** (updates from small random batches) for huge data.
- *Always converges?* Yes (inertia never rises, finitely many assignments), but only to a local optimum.

Self-check

1. Why does inertia always drop as k grows? → More centroids sit closer to points; k = n gives 0.
2. Silhouette with a = 2, b = 6? → 0.67.

TL;DR

k-means = assign to nearest mean, move means, repeat. Scale, k-means++ with restarts, pick k by silhouette/elbow; assumes round clusters.

9.2 Hierarchical (agglomerative) clustering

What it is

Agglomerative clustering starts with every point as a cluster and repeatedly **merges the two closest** until one remains. The merge history is a **dendrogram** — a tree whose height shows the merge distance; cut it at a height to get clusters.

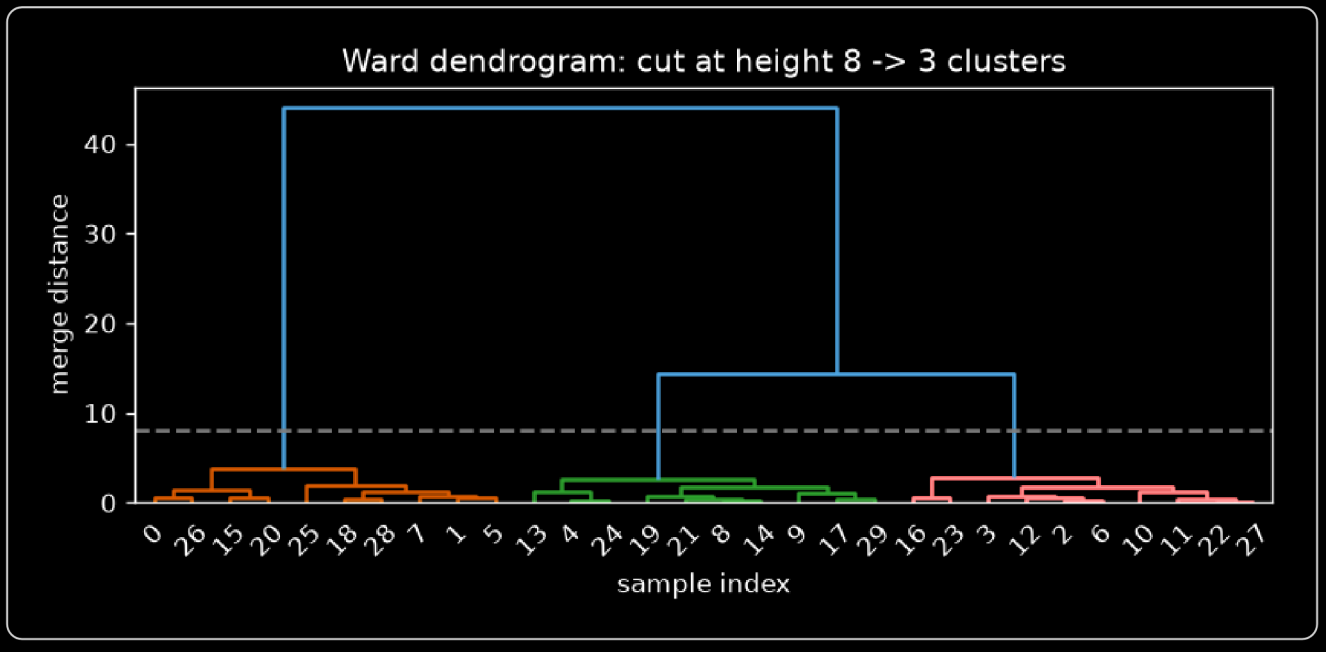
How it works

1. Compute pairwise distances; each point is a cluster.
2. Merge the pair with the smallest **linkage** distance.
3. Update distances; repeat to one cluster.
4. Cut the dendrogram at a height (or request n clusters).

Linkage	Distance between clusters A and B	Behaviour
Single	Closest pair of points	Follows chains — finds snakes and moons, but a thin noise bridge merges clusters ("chaining")
Complete	Farthest pair	Compact clusters of similar diameter
Average	Mean of all pairwise distances	Compromise
Ward (scikit-learn default)	Increase in total within-cluster variance from the merge	Like k-means: round, balanced

```
Z = linkage(X, method="ward")          # SciPy: the full merge history
dendrogram(Z, color_threshold=8)
```

```
last 3 merge heights: [ 3.74 14.37 44.1 ]  (big jump before the final merges = natural cut)
```



Merges below ~4 are within groups; jumps to 14.37 and 44.1 join genuinely different groups. Cutting in the gap (e.g. 8) gives 3 clusters.

Trade-offs

Strengths	Weaknesses
No k in advance — one tree, any number of clusters	$O(n^2)$ memory for distances; slow beyond ~10–20k points
Interpretable picture of nested structure	Greedy merges, never undone
Any metric; linkage adapts to shapes	Noise-sensitive (single) or biased to round shapes (Ward)

Real-world examples

Phylogenetic (evolutionary) trees, clustered gene-expression heatmaps, document/product taxonomies.

In the interview

> **30-second answer:** "Agglomerative clustering starts with each point alone and repeatedly merges the closest pair of clusters, where 'closest' is defined by the linkage — single, complete, average or Ward. The dendrogram records the merges; cutting it at a height gives the clusters. It's great for small data and nested structure, but it's $O(n^2)$ in memory."

- *Linkage for moons?* Single (it chains), if there's little noise between them.

Self-check

1. Dendrogram merge height? → Linkage distance at which the clusters joined.
2. Linkage most like k-means? → Ward.

TL;DR

Merge closest clusters bottom-up; cut the dendrogram at a big gap; linkage decides shapes. Small data only.

9.3 DBSCAN — density-based clustering

What it is

DBSCAN (Density-Based Spatial Clustering of Applications with Noise, Ester et al. 1996) defines clusters as **dense regions separated by sparse ones**: any shape, number found automatically, isolated points labelled **noise** (−1) — like festival crowds versus lone wanderers.

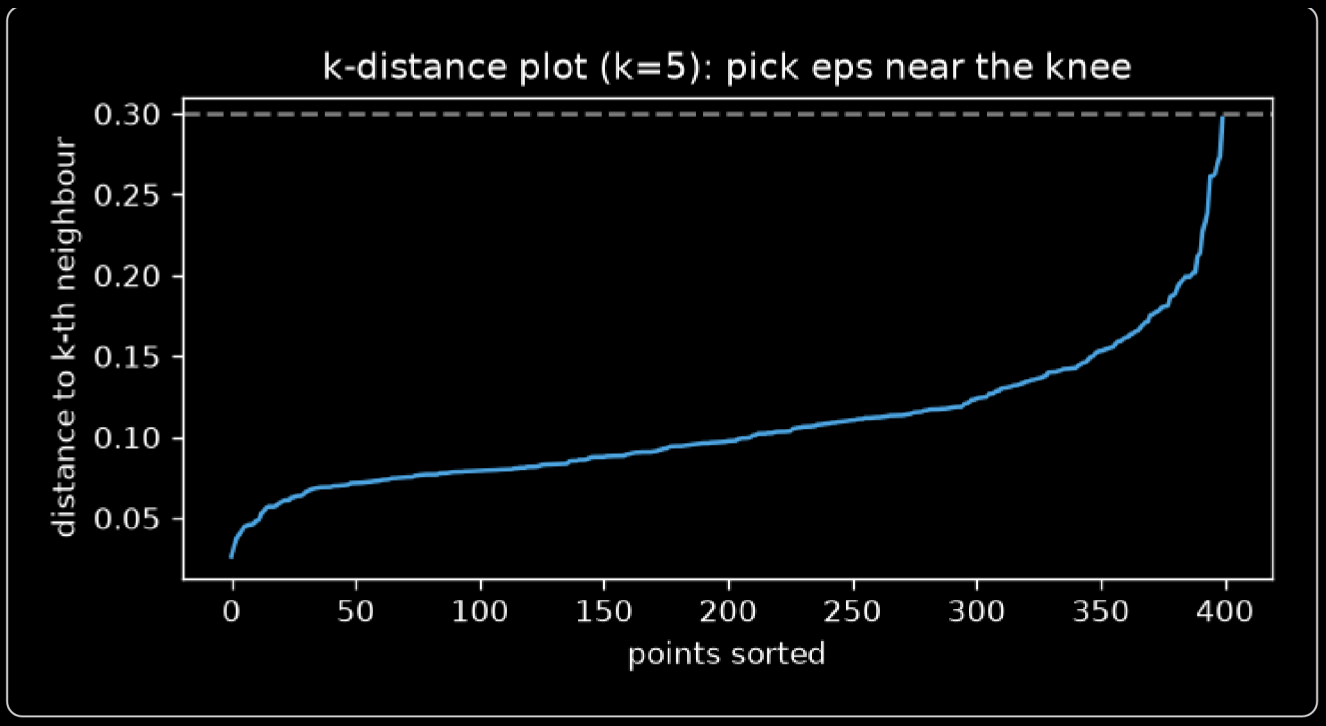
How it works

Parameters **eps** (neighbourhood radius) and **min_samples** (points to count as dense; in scikit-learn including the point itself).

1. **Core point:** ≥ `min_samples` points within eps.
2. **Border point:** within eps of a core point, not core itself.
3. **Noise point:** neither.
4. From an unvisited core point, the cluster is everything **density-reachable** (chains of core points each within eps of the next, plus their border points). Repeat.

Choosing eps: the k-distance plot

Sort each point's distance to its k-th neighbour (k = min_samples) and plot; pick eps near the "knee" where the curve shoots up (dense-region points vs outliers).




```
DBSCAN on moons (min_samples=5)
  eps=0.1  clusters=28  noise points=84
  eps=0.2  clusters=2  noise points=1
  eps=0.3  clusters=2  noise points=0
  eps=0.5  clusters=2  noise points=0
  eps=1.0  clusters=1  noise points=0
```

Too small shatters the data (28 fragments plus noise); too large merges everything. Rule of thumb: min_samples ≈ 2 × number of features (at least 3–5), more for noisy data.

Trade-offs

Strengths	Weaknesses
Any shape; no k	One global density: fails with **different densities**
Built-in noise/outlier detection	Sensitive to eps; hard to choose in high dimensions
Deterministic for core points	Needs scaling; suffers the curse of dimensionality (Chapter 8)

HDBSCAN (`sklearn.cluster.HDBSCAN` since 1.3) handles varying densities and drops eps, needing mainly `min_cluster_size` — often the better modern choice. **OPTICS** is related.

Real-world examples

GPS points → places of interest; geospatial events (crime, taxi pickups); sensor outliers.

In the interview

> **30-second answer:** "DBSCAN groups points in dense regions: a core point has at least min_samples neighbours within eps; clusters are chains of core points plus their border points; everything else is noise. It finds arbitrary shapes and the number of clusters automatically, but struggles with clusters of different densities — HDBSCAN addresses that."

Self-check

1. A tight cluster next to a diffuse one? → One eps can't fit both densities.

TL;DR

Clusters = dense regions; noise labelled −1. eps from a k-distance plot; HDBSCAN for mixed densities.

9.4 Gaussian mixture models (GMM) and EM

What it is

A **Gaussian mixture model** assumes data come from k Gaussians, each with a **mean** (centre), **covariance** (shape and orientation — stretched, tilted ellipses) and **weight** (share of data). Unlike k-means' hard assignment it gives **soft assignments** ("43% A, 35% C, 21% B").

How it works — Expectation-Maximisation (EM), intuitively

We know neither the Gaussians' parameters nor which Gaussian produced each point; knowing one makes the other easy, so EM alternates:

1. **Initialise** means, covariances, weights (scikit-learn: from k-means).
2. **E-step:** each point's **responsibility** per component — P(component | point) by Bayes' rule (weight × density, normalised).
3. **M-step:** re-estimate each Gaussian from all points **weighted by responsibility** (weighted mean, covariance; weight = average responsibility).
4. Repeat until the **log-likelihood** (log-probability of the data under the model) stops improving.

EM never decreases likelihood but finds a local optimum (`n_init` > 1). k-means is the limiting case with hard assignments and identical spherical covariances.

Choosing k: BIC

The **Bayesian Information Criterion** (BIC) = likelihood penalised by parameter count; **lower is better** (`gmm.bic(X)`; also `aic`, Akaike, lighter penalty).

```
GMM model selection by BIC (lower is better)
  k=1  BIC=  4797.3
  k=2  BIC=  4514.9
  k=3  BIC=  4513.3
  k=4  BIC=  4550.7
soft assignments: 103 of 500 points have max membership < 0.9, e.g. [0.43 0.21 0.35]
EM converged in 4 iterations; mixing weights [0.33 0.4  0.27]
```

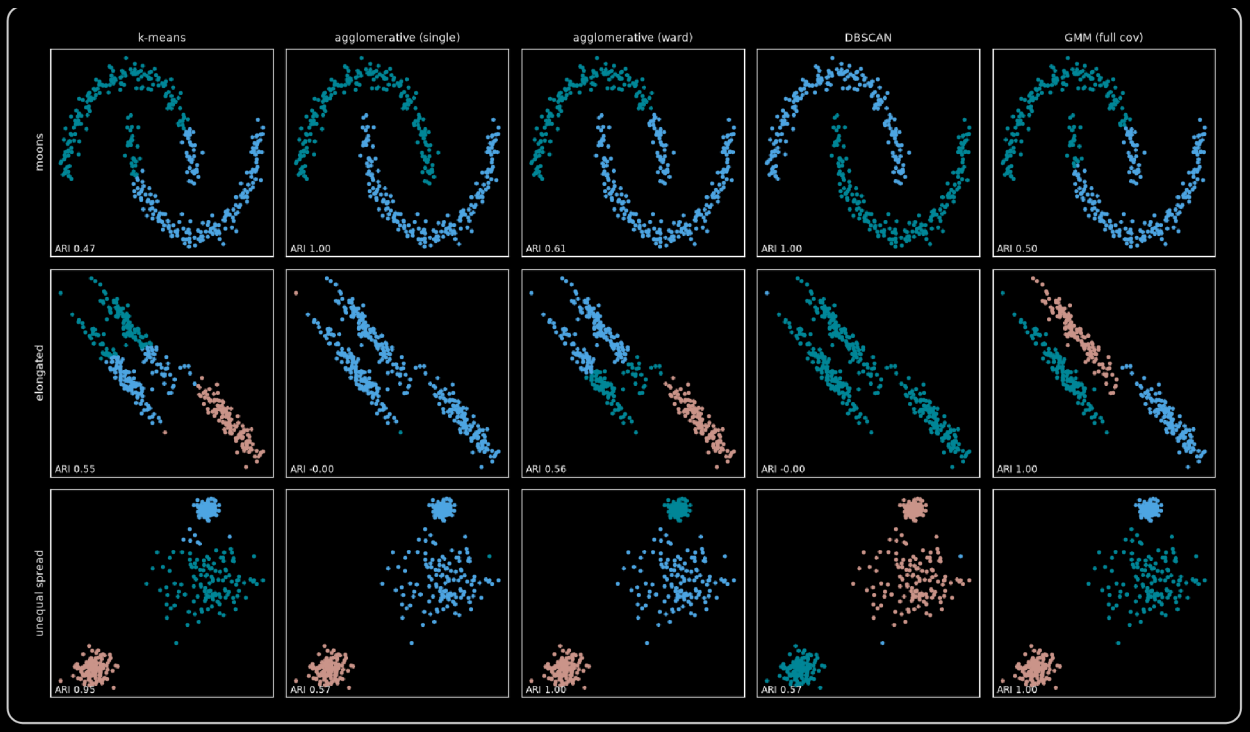
BIC picks k = 3 (the truth), narrowly over 2 because two blobs overlap; ~20% of points are genuinely ambiguous, as the soft assignments honestly show.

`covariance_type`: `"full"` (any ellipse per component), `"tied"` (one shared), `"diag"` (axis-aligned), `"spherical"` (circles, like k-means).

Comparing all four clusterers

`code/clustering_shapes.py` scores each method against the truth with the **Adjusted Rand Index** (ARI: 1 = perfect, ≈ 0 = random):

Adjusted Rand Index	moons	elongated	unequal spread
k-means	0.47	0.55	0.95
agglomerative (single)	1.00	-0.00	0.57
agglomerative (ward)	0.61	0.56	1.00
DBSCAN	1.00	-0.00	0.57
GMM (full cov)	0.50	1.00	1.00



- **Moons** (**non-convex**: a straight line between two cluster points can leave the cluster): only density/chaining methods (DBSCAN, single linkage) succeed.
- **Elongated** and close: only full-covariance GMM models the ellipses; DBSCAN and single linkage merge through the dense bridge.
- **Unequal spread:** GMM and Ward perfect; DBSCAN's one eps can't fit both.

No algorithm wins everywhere — choose by expected shapes, and look at the result.

When to use which clustering algorithm

Algorithm	Needs k?	Cluster shapes	Noise handling	Scales to	Pick it when
k-means	Yes	Round, similar size	None (outliers distort)	Millions (Mini-Batch)	Fast default, compact clusters
Agglomerative	Yes or cut height	Depends on linkage	Poor	~10k	Small data, want a hierarchy
DBSCAN / HDBSCAN	No	Any	Labels noise	~100k+ (with index)	Odd shapes, outliers, unknown k
GMM	Yes (by BIC)	Ellipses, overlapping	None	~100k	Soft probabilities, elongated clusters

In the interview

30-second answer: "A GMM models the data as a weighted mix of Gaussians, each with its own mean and covariance, and fits them with EM: the E-step computes each point's probability of belonging to each component, the M-step re-estimates the parameters with those weights. It gives soft assignments and elliptical clusters; you choose k with BIC. k-means is a special case with hard assignments and spherical clusters."

Self-check

1. E-step computes? → Responsibilities P(component | point).
2. GMM over k-means when? → Elliptical/overlapping clusters, or membership probabilities needed.

TL;DR

GMM = mixture of Gaussians fitted with EM; soft, elliptical clusters; k by BIC (lower better).

9.5 Dimensionality reduction: PCA

What it is

Principal Component Analysis finds new axes — **principal components**, linear combinations of the features — ordered so the first captures the **most variance**, the second the most of the rest while perpendicular, and so on. Keeping the top few compresses data with minimal loss — the most informative "camera angle" on a teapot.

Why it matters

- **Compression / speed** (fewer features); **noise reduction** (small-variance directions are often noise); **visualisation** in 2-D/3-D.
- **Removes correlation** (multicollinearity): components are uncorrelated.
- Fights the **curse of dimensionality** before distance-based methods.

How it works

(Eigenvectors and SVD: Chapter 2, section 2.3.)

1. **Centre** each feature; **standardise** if units differ.
2. Compute the **covariance matrix** C (d × d).
3. Its **eigenvectors** are the directions; **eigenvalues** the variance along each ($C \cdot v = \lambda \cdot v$).
4. Sort by eigenvalue; **explained variance ratio** of component i = λ_i / $\Sigma\lambda$.
5. **Project:** Z = X_centred · V_k (top-k eigenvectors as columns).

Libraries compute this via the **SVD** of the centred data, which is more numerically stable; scikit-learn does.

Worked example — hours studied vs exam score, 5 students (`code/dim_reduction.py`):

```
X = np.array([[2, 50], [4, 60], [6, 70], [8, 85], [10, 95]], dtype=float)
Xs = (X - X.mean(axis=0)) / X.std(axis=0, ddof=1)   # centre and scale
C = np.cov(Xs, rowvar=False)                          # covariance matrix
vals, vecs = np.linalg.eigh(C)                        # eigen-decomposition
z = Xs @ vecs[:, 0]                                   # project onto PC1
```

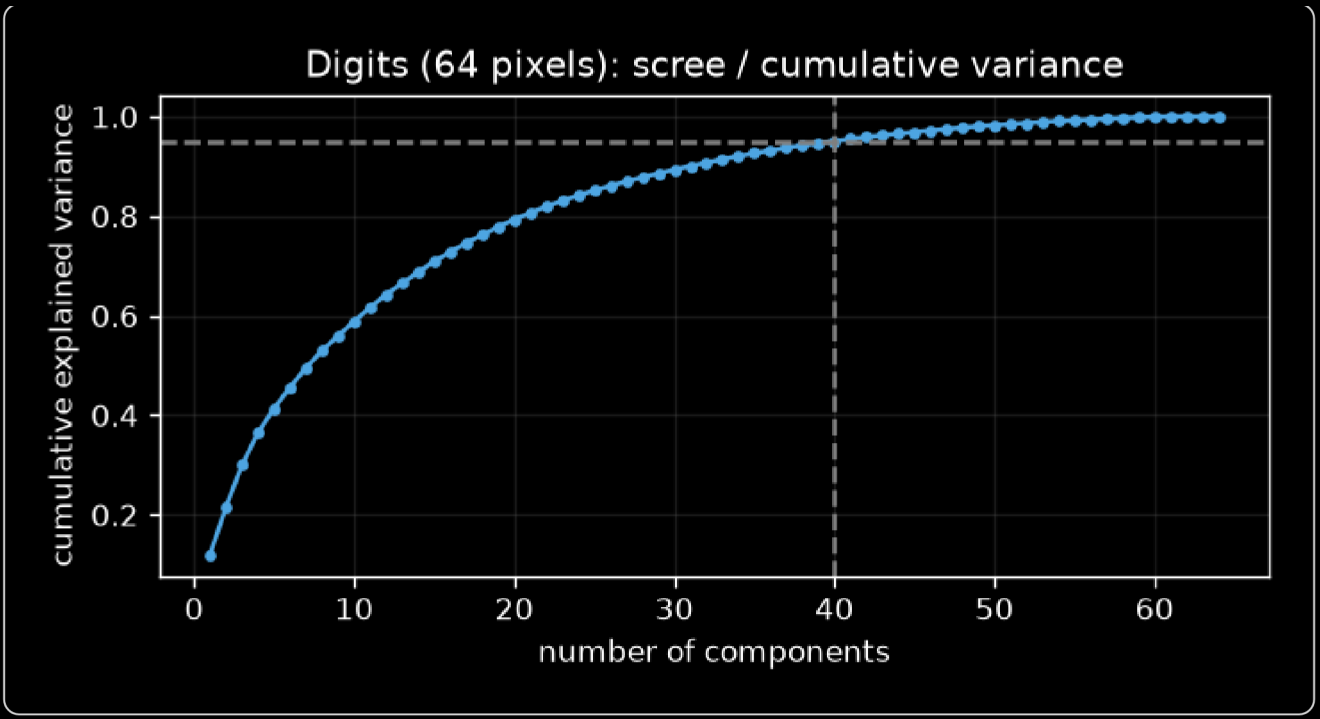
```
covariance (standardised):
 [[1.    0.997]
 [0.997 1.   ]]
eigenvalues: [1.997 0.003]  explained variance ratio: [0.999 0.001]
PC1 direction: [0.707 0.707]
students on PC1: [-1.75 -0.91 -0.08  0.95  1.79]
sklearn explained_variance_ratio_: [0.999 0.001]
```

For two standardised features with correlation r, eigenvalues are always 1 ± r with directions (1, ±1)/√2. Here r = 0.997, so PC1 ("overall diligence") holds 99.9%: two columns compress to one number per student. (Eigenvector sign is arbitrary; flipped so bigger = more.)

How many components? Explained variance

8×8 digit images (64 pixels):

```
components for 50% variance:  8 of 64
components for 80% variance: 21 of 64
components for 90% variance: 31 of 64
components for 95% variance: 40 of 64
components for 99% variance: 54 of 64
PCA(n_components=0.95) keeps 40 components, reconstruction MSE = 0.047
```



Keep 90–99% of variance (`PCA(n_components=0.95)`), find an elbow in the **scree plot** (eigenvalue vs component number), or cross-validate the count with the downstream model. `inverse_transform` maps back; the **reconstruction error** shows what was lost.

Variants

- **Incremental PCA:** data too big for memory (mini-batches).
- **Randomized PCA** (`svd_solver="randomized"`): fast approximate top components.
- **Kernel PCA:** non-linear via the kernel trick (Chapter 8).
- **TruncatedSVD:** PCA-like for sparse matrices (e.g. word counts), no centring.

Trade-offs / when not to use it

- **Linear** only — curved **manifolds** (surfaces the data lies on, like a "Swiss roll") need non-linear methods.
- Components mix all features → hard to interpret; sensitive to scaling and outliers.
- Max variance ≠ most predictive: a low-variance direction may separate classes (supervised alternative: **LDA**, linear discriminant analysis, picks directions separating known classes).

Common mistakes

- Not standardising mixed units (the large-unit feature becomes PC1).
- Fitting PCA before splitting (leakage) — put it in the pipeline.
- Over-reading a 2-component plot explaining only 20% of variance.

Real-world examples

Eigenfaces (Turk & Pentland, 1991); compressing embeddings before vector search; population-structure plots in genetics; decorrelating features before regression; image compression.

In the interview

30-second answer: "PCA centres (and usually scales) the data, computes the covariance matrix, and takes its eigenvectors ordered by eigenvalue. Each eigenvalue is the variance along that direction, so the explained-variance ratio tells you how much information each component keeps. You project onto the top k — chosen for, say, 95% variance — to compress, denoise or visualise. It's linear, unsupervised, and sensitive to scaling."

- *PCA vs SVD?* PCA is usually computed via SVD of centred data: right singular vectors = principal directions; squared singular values/(n−1) = eigenvalues.
- *Helps every model?* No — trees don't need it, and it can drop low-variance predictive directions.

Self-check

1. Two standardised features, correlation 0.8: PC1's explained variance ratio? → (1 + 0.8)/2 = 0.9.

TL;DR

PCA rotates data onto uncorrelated axes of decreasing variance; keep the top ones. Scale first; choose k by explained variance; linear only.

9.6 t-SNE and UMAP — non-linear visualisation

What they are

t-SNE (t-distributed Stochastic Neighbor Embedding, van der Maaten & Hinton, 2008) and **UMAP** (Uniform Manifold Approximation and Projection, McInnes et al., 2018) are **non-linear** methods mainly for **visualising** high-dimensional data in 2-D/3-D, keeping **neighbours as neighbours**.

How t-SNE works (intuition)

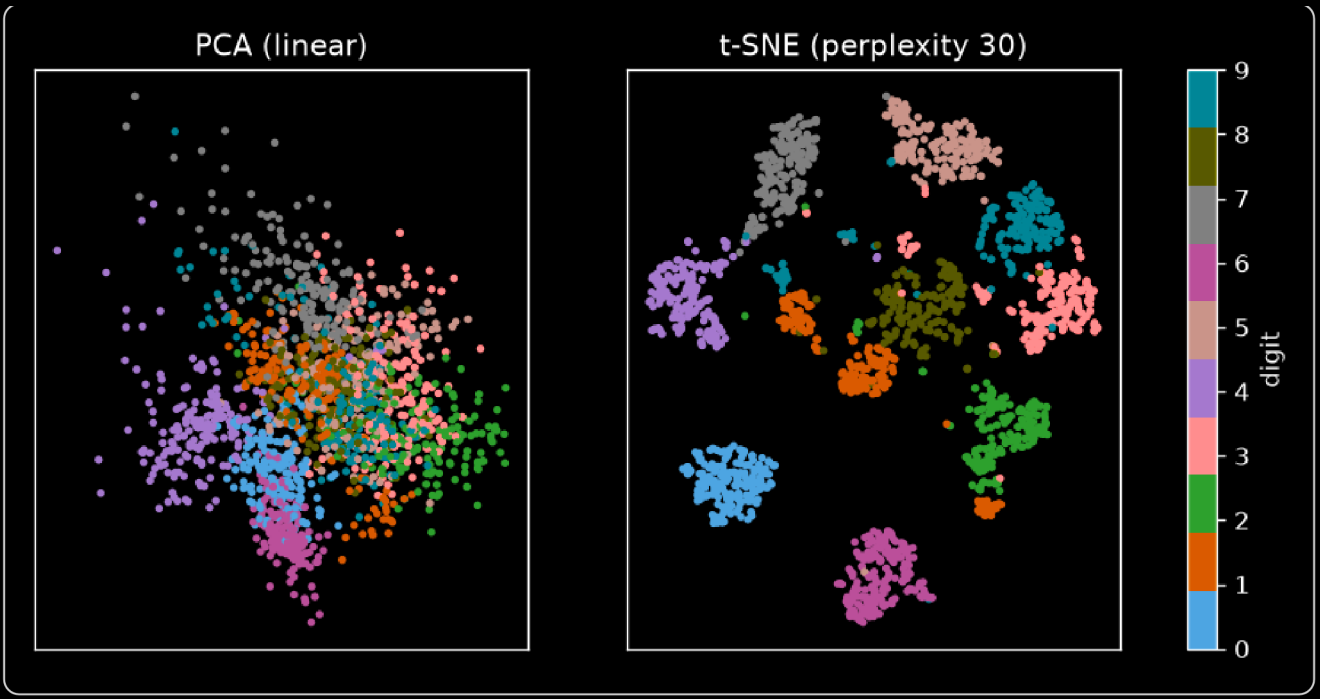
1. In the original space, turn distances into neighbour probabilities (Gaussian kernel); **perplexity** (typically 5–50) ≈ how many neighbours each point considers.
2. In 2-D, use a heavy-tailed **Student-t** distribution, letting moderately distant points push far apart instead of crowding the middle.
3. Move 2-D points by gradient descent to match the two distributions (minimise **KL divergence**).

UMAP builds a weighted k-nearest-neighbour graph (`n_neighbors`) and optimises a low-dimensional layout matching it (`min_dist` controls packing). Usually faster than t-SNE on large data, keeps more global arrangement, and can `transform` new points.

Code — PCA vs t-SNE on digits

```
Z_pca = PCA(n_components=2, random_state=0).fit_transform(Xs)
Z_tsne = TSNE(n_components=2, perplexity=30, init="pca", random_state=0).fit_transform(
    PCA(n_components=30, random_state=0).fit_transform(Xs))   # common trick: PCA to ~30-50 dims first
print(trustworthiness(Xs, Z, n_neighbors=5))
```

```
trustworthiness (k=5) PCA  : 0.818
trustworthiness (k=5) t-SNE: 0.992
```



PCA's best linear view mixes most digits; t-SNE separates all ten into islands without seeing labels (colour is only for the plot). **Trustworthiness** (0–1) = how many map neighbours were true neighbours.

UMAP (`umap-learn` not installed here — not run):

```
# pip install umap-learn   (not run)
import umap
reducer = umap.UMAP(n_neighbors=15, min_dist=0.1, random_state=0)
Z = reducer.fit_transform(Xs)
Z_new = reducer.transform(X_new_scaled)   # UMAP can embed new points; sklearn's TSNE cannot
```

Reading t-SNE/UMAP plots safely

- **Cluster sizes mean nothing** (t-SNE expands dense clusters, contracts sparse ones).
- **Inter-cluster distances mean little.**
- **Results change with perplexity/n_neighbors and seed** — try several.
- **Don't cluster on t-SNE output** or use it as features — it's for looking. (UMAP is sometimes used before clustering, with care.)

PCA vs t-SNE vs UMAP

	PCA	t-SNE	UMAP
Type	Linear	Non-linear	Non-linear
Preserves	Global variance	Local neighbourhoods	Local, plus more global than t-SNE
Speed	Very fast	Slow on big data (Barnes-Hut, O(n log n))	Fast
New points	`transform`	No (in scikit-learn)	`transform`
Deterministic	Yes	No (seed)	No (seed)
Use for	Compression, pre-processing, denoising, quick look	2-D cluster visualisation	Visualisation, sometimes pre-clustering
Key knobs	n_components	perplexity	n_neighbors, min_dist

Rule of thumb: PCA first (fast, interpretable, good pre-step to ~50 dims); t-SNE or UMAP to **look**; UMAP for large data or embedding new points.

In the interview

> **30-second answer:** "PCA is a linear projection that keeps global variance — good for compression and pre-processing. t-SNE and UMAP are non-linear and keep local neighbourhoods, so they're great for 2-D visualisation of clusters, but cluster sizes and inter-cluster distances in the plot aren't meaningful and the output depends on hyperparameters. UMAP is faster and can transform new data."

Self-check

1. Which can embed new points out of the box? → PCA and UMAP.

TL;DR

PCA to compress; t-SNE/UMAP to see. Only local neighbourhoods in their plots are trustworthy.

9.7 Anomaly detection

What it is

Anomaly (outlier) detection finds points differing markedly from the majority: fraud, failing machines, intrusions, data-entry errors. Anomalies are usually **rare and unlabelled**, so it is mostly unsupervised. scikit-learn distinguishes **outlier detection** (training data contains the outliers) from **novelty detection** (train on clean data, flag new points).

Statistical baseline: z-score and robust z-score

z-score z = (x − mean)/std, flag |z| > 3 — but the outlier inflates mean and std (**masking**). Robust version: median and **MAD** (median absolute deviation), modified z = 0.6745·(x − median)/MAD, flag |modified z| > 3.5 (Iglewicz & Hoaglin). The box-plot **IQR rule** (outside Q1 − 1.5·IQR, Q3 + 1.5·IQR) is also robust.

```
amount   z-score   robust z
    42     -0.32      -0.54
    ...
    60     -0.25       1.89
   950      3.32     121.95  <- anomaly
mean=121.7 (dragged up by the outlier), median=46.0
```

The $950 payment barely passes z = 3: with population std, no point in a sample of n can exceed |z| = √(n − 1) — here √11 = 3.32 — so small samples **cannot** flag extremes reliably. The robust z says 122. These rules are univariate; for many features, use the methods below.

Isolation Forest

Idea (Liu, Ting & Zhou, 2008): anomalies are **few and different, so easy to isolate**. Random trees split on a random feature at a random value between its min and max; normal points in dense regions need many splits, anomalies few.

1. Build ~100–200 **isolation trees**, each on a small subsample (default 256 rows).
2. Measure each point's average **path length** to its leaf.
3. Convert to a score in (0, 1): near 1 = anomaly, ≤ ~0.5 = normal.

```
anomaly score of (6,6): 0.82   typical point (median): 0.43
```

(scikit-learn's `score_samples` returns the *negative*: lower = more abnormal.) Linear time, scales to large high-dimensional data, no distances, no scaling. Weakness: axis-parallel splits can miss anomalies visible only in a *combination* of features.

One-Class SVM

Learns a boundary around normal data (Schölkopf et al., 2001); with an RBF kernel, a smooth region holding most points; outside = anomaly. **ν** (`nu`) upper-bounds the fraction of training outliers (and lower-bounds the fraction of support vectors). Good for **novelty detection** on clean data; sensitive to γ and ν; needs scaling; poor past tens of thousands of points (`SGDOneClassSVM` is a scalable linear alternative).

Local Outlier Factor (LOF)

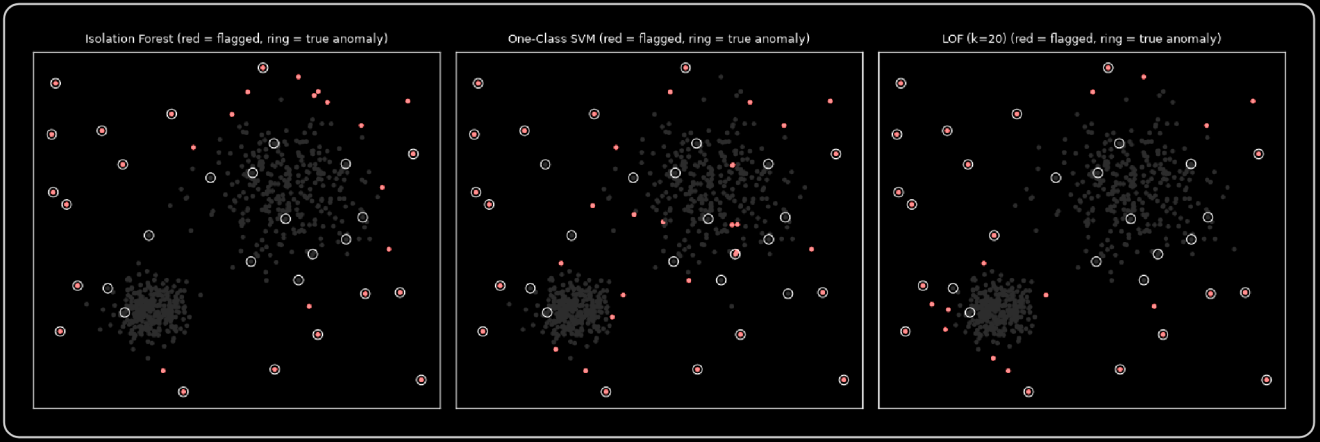
Compares a point's **local density** with its k neighbours' (Breunig et al., 2000): LOF ≈ 1 normal; LOF ≫ 1 much sparser than neighbours (outlier). Being **local**, it catches a point just outside a *tight* cluster that would look normal near a *diffuse* one. Scores training data by default; `novelty=True` scores new points.

Comparison

Tight cluster, diffuse cluster, 30 uniformly scattered anomalies (`code/anomaly_detection.py`):

```
detectors = {
    "Isolation Forest": IsolationForest(n_estimators=200, contamination=30 / 630, random_state=0),
    "One-Class SVM": OneClassSVM(kernel="rbf", gamma=0.3, nu=0.05),
    "LOF (k=20)": LocalOutlierFactor(n_neighbors=20, contamination=30 / 630),
}
pred = det.fit_predict(X)           # +1 inlier, -1 outlier
```

```
detector            ROC-AUC   avg precision   flagged   true anomalies caught
Isolation Forest      0.920           0.628        30   17 / 30
One-Class SVM         0.813           0.522        36   16 / 30
LOF (k=20)            0.886           0.617        30   18 / 30
```



About a third of the "anomalies" landed **inside** normal clusters — nothing can (or should) find those, so recall tops out near 18/30. `contamination` (expected outlier fraction) fixes the threshold; in practice you rarely know it, so rank by score and review the top. With a few labels, evaluate with **precision–recall** (average precision): anomalies are rare and ROC-AUC flatters under heavy imbalance.

Method	Idea	Strengths	Weaknesses
z-score / MAD / IQR	Distance from centre in std/MAD units	Trivial, explainable	One feature at a time; one "normal" centre
Isolation Forest	Anomalies isolate in few random splits	Fast, scales, high-dim, no scaling	Misses combination-only anomalies
One-Class SVM	Boundary around normal data	Flexible shapes, good for novelty	Slow on big n; γ, ν tuning; scaling
LOF	Density vs neighbours' density	Clusters of different density	O(n²)-ish neighbour search; choice of k; scaling

Also: **autoencoders** (flag high reconstruction error; neural networks, later chapters), Elliptic Envelope (robust Gaussian fit), and time-series methods for **seasonal** data (repeating daily/weekly/yearly patterns).

Real-world examples

Card-fraud scoring, failing disks or turbines from sensor logs, network intrusion detection, data-quality checks before training.

In the interview

> **30-second answer:** "For one feature, a robust z-score or IQR rule is enough. For multivariate data, Isolation Forest is my default: anomalies get isolated by random splits in fewer steps, so short average path length means anomaly — it's fast and scales. LOF compares local density to neighbours and handles clusters of different density. One-Class SVM learns a boundary around clean data for novelty detection. Evaluate with precision–recall on whatever labels exist, and set the threshold from business cost."

- *Why not plain z-scores?* Masking (outlier inflates mean/std), and univariate.
- *Isolation Forest intuition?* Few-and-different points are cut off by random splits quickly.

Self-check

1. Isolation Forest: short or long path = anomaly? → Short.
2. LOF 1.0? → Same density as neighbours — normal.
3. Why is ROC-AUC misleading for rare anomalies? → A small false-positive *rate* over many negatives still means many false alarms.

TL;DR

Robust statistics for one feature; Isolation Forest as multivariate default; LOF for varying densities; One-Class SVM for novelty. Rank by score; judge with precision–recall.

9.8 Association rules (Apriori)

What it is

Association rule mining finds if-then patterns in transactions (*{diapers}* → *{beer}*), classically **market-basket analysis**.

Measure	Formula	Meaning
Support(A)	Fraction of baskets containing A	How common
Confidence(A → B)	support(A ∪ B) / support(A)	How often B appears with A
Lift(A → B)	confidence(A → B) / support(B)	> 1: A makes B more likely; = 1 independent; < 1 less likely

How Apriori works

Apriori principle (anti-monotone): every subset of a frequent **itemset** must be frequent, so if {beer} is rare no superset is counted.

1. Count single items; keep support ≥ min_support (L1).
2. **Join** frequent (k−1)-itemsets into k-item candidates; **prune** any with an infrequent (k−1)-subset.
3. Count candidates in one pass; keep frequent ones (Lk); repeat until none.
4. Generate rules A → B from each frequent itemset; keep confidence ≥ min_confidence; rank by lift.

`code/apriori.py`, five baskets (min_support 0.6, min_confidence 0.7):

```
L1: {beer}=0.6, {bread}=0.8, {diapers}=0.8, {milk}=0.8
L2: {beer,diapers}=0.6, {bread,diapers}=0.6, {bread,milk}=0.6, {diapers,milk}=0.6

rule                        support  confidence  lift
           beer -> diapers       0.6        1.00  1.25
         diapers -> beer         0.6        0.75  1.25
           bread -> diapers      0.6        0.75  0.94
           bread -> milk         0.6        0.75  0.94
           ...
```

Every beer basket has diapers (confidence 1.0, lift 1.25): a real association. bread → milk has 75% confidence but lift 0.94 — milk is in 80% of baskets anyway, so bread makes it slightly *less* likely. **High confidence alone misleads — check lift.** (The only 3-item candidate, {bread, diapers, milk}, has support 0.4 and is dropped.)

FP-Growth finds the same itemsets without candidates (compressing data into a prefix tree storing shared basket beginnings once), usually faster; `mlxtend` provides both.

Real-world examples and limits

"Frequently bought together", product placement, cross-selling, co-occurring symptoms or page visits. (The beer-and-diapers story is partly legend.) Limits: many mostly trivial rules, arbitrary thresholds, association ≠ causation; modern recommenders mostly use collaborative filtering and embeddings.

In the interview

> **30-second answer:** "Association rules find co-occurrence patterns like A → B. Support is how common an itemset is, confidence is P(B | A), and lift is confidence divided by B's base rate — lift above 1 means a genuine positive association. Apriori finds frequent itemsets level by level and prunes candidates using the fact that all subsets of a frequent set must be frequent."

Self-check

1. Support(A∪B) = 0.2, support(A) = 0.4, support(B) = 0.5? → Confidence 0.5, lift 1.0 (independent).

TL;DR

Support, confidence, lift; Apriori prunes by "subsets of frequent sets are frequent". Always check lift.

9.9 Evaluating unsupervised models

The problem

There is no **ground truth** to compare against, and clustering **always returns clusters**, even from noise — so evaluate from several angles.

Metrics

Kind	Metric	Range / better	Notes
Internal (no labels)	Silhouette	−1 to 1, higher	Cohesion vs separation; favours round, separated clusters
	Davies–Bouldin	≥ 0, **lower**	Average similarity of each cluster to its most similar one
	Calinski–Harabasz	≥ 0, higher	Between- vs within-cluster dispersion
	Inertia / BIC	Lower	Only to compare k within one algorithm
External (some labels)	Adjusted Rand Index (ARI)	≈ 0 random, 1 perfect	Pair-counting agreement, chance-corrected
	Normalized Mutual Information (NMI)	0 to 1, higher	Shared information between labelings
Dimensionality reduction	Explained variance, reconstruction error	Higher / lower	PCA-type methods
	Trustworthiness	0 to 1, higher	Are map neighbours true neighbours?
Anomaly detection	Precision@k (real anomalies in the top k flagged), average precision (small labelled set)	Higher	Rank-based; matches how analysts review alerts

Internal metrics can disagree with reality

`code/evaluate_clustering.py`:

```
iris (true k = 3 species)
  k-means k=2          silhouette= 0.582  DB= 0.59  CH=  251.3  ARI= 0.57  NMI= 0.73
  k-means k=3          silhouette= 0.460  DB= 0.83  CH=  241.9  ARI= 0.62  NMI= 0.66
  k-means k=4          silhouette= 0.387  DB= 0.87  CH=  207.3  ARI= 0.47  NMI= 0.60

moons (true k = 2, non-convex)
  k-means k=2          silhouette= 0.497  DB= 0.81  CH=  556.6  ARI= 0.47  NMI= 0.38
  DBSCAN eps=0.3       silhouette= 0.385  DB= 1.03  CH=  342.5  ARI= 1.00  NMI= 1.00
```

- Iris: all internal metrics prefer **k = 2** (two species overlap heavily, so "2 blobs" is geometrically cleaner); biology says 3.
- Moons: internal metrics prefer k-means' wrong split over DBSCAN's perfect one, since all three reward **convex** clusters.

A practical evaluation checklist

1. **Look at it:** plot clusters on a PCA/UMAP projection; read samples per cluster.
2. **Internal metrics** to compare settings of the *same* algorithm (k, eps), knowing their shape bias.
3. **Stability:** re-run with seeds or bootstrap resamples — do clusters persist (ARI between runs)?
4. **External check:** a small labelled sample or known categories, via ARI/NMI.
5. **Downstream value** — the real test: better campaigns, better model features, flagged anomalies confirmed as fraud?

In the interview

> **30-second answer:** "Without labels I use internal metrics like silhouette, Davies–Bouldin and Calinski–Harabasz to compare settings — but they favour convex clusters. If some labels exist, ARI or NMI. I check stability across seeds and, most importantly, whether the result is useful for the downstream task. For dimensionality reduction: explained variance, reconstruction error, trustworthiness; for anomalies: precision on a reviewed sample."

Self-check

1. Why can silhouette prefer the wrong answer on moons? → It rewards compact, convex, separated clusters.

TL;DR

Internal metrics (shape-biased) + external labels when available + stability + downstream usefulness. Always look at the results.

Chapter TL;DR

- **k-means:** assign to nearest centroid, recompute means, repeat; k-means++ + restarts; k by silhouette/elbow; round clusters; scale.
- **Hierarchical:** merge closest clusters bottom-up; dendrogram; linkage sets shape; small data.
- **DBSCAN:** dense regions = clusters, sparse points = noise; any shape, no k; eps from a k-distance plot; HDBSCAN for varying density.
- **GMM + EM:** soft, elliptical clusters; E-step responsibilities, M-step weighted updates; k by BIC.
- **PCA:** covariance eigenvectors = max-variance directions; keep ~95% variance; linear; scale first.
- **t-SNE / UMAP:** non-linear neighbourhood-preserving maps for visualisation; ignore cluster sizes and gaps; UMAP faster, transforms new data.
- **Anomalies:** robust z/IQR univariate; Isolation Forest default; LOF for local density; One-Class SVM for novelty; precision–recall.
- **Apriori:** support, confidence, lift; subset pruning; trust lift over confidence.
- **Evaluation:** internal metrics are shape-biased — combine with external labels, stability and downstream usefulness.